

基于微信小程序的校园个人物品租赁与交易系统设计与实现

论文参考稿

选题方向	软件工程 / 信息管理与信息系统 / 计算机科学与技术
项目来源	LeaseTransaction 仓库中的校园个人物品租赁与交易系统
技术路线	微信小程序 + Vue 3 + Spring Boot 3.x + MyBatis-Plus + MySQL + Redis
文档用途	作为毕业论文或课程设计论文的结构、表达和技术内容参考
使用说明	提交前需替换为学校模板要求的封面、摘要格式、参考文献样式与真实测试数据

摘要

随着高校在校生规模持续扩大，学生之间的闲置物品流转需求和短期租赁需求日益明显。传统二手交易平台虽然能够覆盖广泛用户，但在校园场景中仍然存在身份核验弱、同校近场交易不便、租赁流程不完整、押金与信用风险高等问题。针对上述痛点，本文围绕校园个人物品租赁与交易系统的设计与实现展开研究，提出一种基于微信小程序的轻量化平台方案。该方案以校园真实身份为信任基础，整合物品发布、租赁、出售、订单跟踪、实时消息、信用评价和后台监管等功能，构建适用于高校场景的可信交易闭环。

系统总体上采用前后端分离架构，用户端基于微信小程序实现，管理端采用 Vue 3 与 TypeScript 构建，服务端基于 Spring Boot 3.x、MyBatis-Plus、MySQL 与 Redis 完成核心业务支撑。为了同时兼顾租赁和交易两种业务模式，系统设计了统一订单模型，并结合押金冻结、归还验收、信用评分和异常申诉等机制，提高业务处理的完整性与安全性。本文进一步从需求分析、系统架构、数据库设计、核心流程、性能优化及测试方案等方面对系统进行了详细论述。

研究表明，该系统能够有效提升校园闲置物品利用率，降低学生获取低频用品的成本，并为高校场景下的数字化资源共享提供可落地的技术实现路径。本文形成的设计思路和实现方案可为同类校园服务系统的建设提供参考。

关键词：微信小程序；校园租赁；二手交易；Spring Boot；订单系统；信用评价

Abstract

With the increasing scale of higher education, students have growing demands for idle-item circulation and short-term leasing on campus. General second-hand platforms can cover broad user groups, yet they still suffer from weak identity verification, inefficient nearby transactions, incomplete leasing workflows, and limited risk control in campus scenarios. To address these issues, this paper designs and implements a campus-oriented personal item leasing and trading system based on WeChat Mini Program. The solution builds trust on verified campus identities and integrates item publishing, leasing, selling, order tracking, instant messaging, credit evaluation, and administrative supervision into a unified service loop.

The system adopts a separated front-end and back-end architecture. The client side is implemented with WeChat Mini Program, the administrative console is built with Vue 3 and TypeScript, and the server side is powered by Spring Boot 3.x, MyBatis-Plus, MySQL, and Redis. To support both leasing and direct trading, a unified order model is introduced together with deposit freezing, return inspection, credit scoring, and dispute handling mechanisms. This paper describes the project from requirement analysis, architecture design, database design, key implementation details, to performance considerations and testing strategy.

The resulting design can improve the utilization rate of campus idle resources, reduce the cost of obtaining low-frequency items, and provide a practical reference for building trustworthy digital sharing services in university environments.

Key words: WeChat Mini Program; campus leasing; second-hand trading; Spring Boot; order management; credit evaluation

1 绪论

1.1 研究背景

高校学生群体具有流动性强、预算敏感、对低频用品需求波动明显等特征。教材、数码配件、摄影器材、运动器材和宿舍小家电等物品在校园中存在大量闲置，单纯依赖线下社群或综合电商平台难以兼顾效率与安全。一方面，传统平台多强调跨区域广覆盖，无法充分利用校园近场交付优势；另一方面，租赁业务要求对物品状态、时间区间、押金和归还责任进行精细化管理，这也超出了普通二手交易平台的标准能力边界。

微信小程序具备无需安装、入口便捷、社交触达成本低等特点，特别适合构建高频轻量服务。若能将微信生态的登录、支付和消息触达能力与校园身份认证、信用评价模型结合起来，便可以在有限开发成本下形成符合高校场景特征的可信交易系统。因此，围绕校园个人物品租赁与交易平台展开研究，具有较强的应用价值与实践意义。

1.2 研究目的与意义

本课题旨在设计并实现一套覆盖物品发布、租赁预约、购买下单、支付结算、消息沟通、信用评价和后台监管的校园交易平台，通过系统化手段提升校园资源复

用效率。其理论意义在于对校园场景下“交易 + 租赁”融合模式进行信息系统建模；其实践意义在于降低大学生的使用成本、缩短供需匹配时间，并为高校绿色低碳资源共享提供数字化工具。

1.3 国内外相关研究概述

现有研究多围绕电子商务平台、共享经济平台与校园综合服务平台展开。国外研究更重视平台治理、信用机制与用户行为建模，国内研究则更多关注二手交易系统、小程序应用开发和校园场景业务整合。总体来看，专门针对校园内部短周期租赁与近场交付的系统研究仍相对不足，尤其在统一订单建模、押金风险控制和校园身份可信核验方面缺乏细化设计。本文结合真实项目需求对这些关键问题进行针对性补充。

1.4 研究内容与论文结构

本文首先分析校园租赁与交易业务的实际需求，随后给出系统总体架构设计与数据库设计方案，在此基础上对身份认证、物品管理、统一订单模型、实时消息和信用评价等关键模块展开说明，最后结合测试与优化策略对系统的可用性与扩展性进行总结。整篇论文可作为毕业论文、课程设计报告或项目答辩材料的基础参考文本。

2 需求分析

2.1 业务场景分析

系统的目标用户主要包括普通学生用户、平台管理员和潜在的校园组织运营者。学生用户既可能是物品发布者，也可能是租赁方或购买方；管理员负责审核用户身份、物品内容和异常订单；运营者则关注活跃度、类别热度和资金流向等经营性指标。系统需要同时支撑短期租赁、直接出售、线下自提、校内配送以及消息协商等多种业务场景。

2.2 功能需求分析

模块	核心功能	业务价值
用户管理	微信授权登录、校园身份核验、个人资料维护、信	建立真实可信的校园用户体系

	用积分展示	
物品管理	物品发布、编辑、上下架、图片上传、分类标签、价格与押金设置	提升供给信息质量和可检索性
浏览搜索	关键词检索、分类筛选、价格区间筛选、附近推荐、排序展示	提高供需匹配效率
订单系统	租赁预约、购买下单、支付确认、归还验收、退款处理、异常申诉	支撑双模式业务闭环
消息与评价	即时沟通、订单通知、评价反馈、信用记录	增强交易透明度和用户信任
管理后台	用户审核、物品审核、订单监控、公告配置、数据统计	保障平台治理和运营可控

从功能边界看，租赁流程是本系统区别于普通二手平台的核心能力。租赁订单不仅需要记录租期和金额，还必须处理押金冻结、到期提醒、归还确认和损坏赔偿等状态迁移。因此系统在模型设计时不能简单复用交易订单，而应构建能够容纳多阶段状态流转的统一订单实体。

2.3 非功能需求分析

在可用性方面，系统应遵循小程序轻量、操作路径短和信息反馈清晰的设计原则，使用户能够在有限页面深度内完成搜索、下单和沟通。在安全性方面，系统需要保证身份信息、订单数据和支付回调的完整性，防止越权访问、重复支付和恶意刷单。在性能方面，热门物品列表、分类统计和首页推荐等接口需要具备较好的响应能力，并通过缓存和索引设计降低数据库压力。在可维护性方面，系统架构应支持前后端分离、模块化扩展和配置化运营，以满足后续功能迭代需要。

3 系统总体设计

3.1 技术架构设计

系统采用典型的三层分离思路。表现层由微信小程序和 Vue 3 管理后台组成，小程序面向校园普通用户提供交易入口，后台面向管理员提供审核与统计能力。业务层由 Spring Boot 3.x 服务承载，通过控制器、服务层、数据访问层实现分层解耦。

数据层使用 MySQL 存储核心业务数据，Redis 缓存高频访问数据和会话类信息，对象存储或本地文件系统负责图片资源保存。

层次	主要技术	职责说明
用户层	微信小程序、Vue 3、TypeScript、TailwindCSS	完成交互展示、表单提交、状态呈现和后台管理
业务层	Spring Boot 3.x、Spring Security、WebSocket	提供认证授权、订单处理、消息推送和业务编排
数据层	MySQL 8.0、Redis、对象存储	存储业务数据、缓存热点数据和管理文件资源
外部集成	微信登录、微信支付、腾讯地图	支持身份登录、支付结算和附近推荐

3.2 功能架构设计

按照职责划分，系统可以抽象为用户中心、物品中心、订单中心、支付中心、消息中心、信用中心和运营中心七个子域。用户中心负责登录鉴权和实名状态管理；物品中心管理发布、检索和审核；订单中心处理租赁与交易流程；支付中心负责押金、租金和货款的结算协调；消息中心用于订单通知和即时沟通；信用中心记录评价与违规行为；运营中心则面向管理员提供配置和数据分析功能。这种拆分方式既能够覆盖当前业务，也有利于后续服务化演进。

3.3 核心业务流程设计

在租赁场景中，用户浏览物品后选择租赁时间段并提交申请，系统首先校验库存状态和时间区间占用情况，随后生成待支付订单。支付成功后，订单进入待交付状态；双方完成交付后进入租赁中状态；系统根据约定时间进行归还提醒；归还后由物品所有者或管理员执行验收，若无异常则完成租金结算并退回剩余押金。交易场景则省略归还节点，整体流程更接近标准电商订单，但仍可复用统一订单模型的支付、确认和评价能力。

4 数据库设计

4.1 数据模型设计思路

数据库设计以用户、物品、订单为主线，辅以支付、评价、消息和信用记录等辅助实体。为了兼顾租赁与交易两种业务模式，订单表中引入业务类型字段、租赁起止时间、押金金额、验收状态和退款状态等扩展字段，通过状态机描述不同模式下的流转规则。该方式能够避免为每类业务重复设计完全独立的订单结构，同时便于统一统计和运维管理。

4.2 主要数据表设计

数据表	关键字段	说明
user	openid, student_no, campus, credit_score, auth_status	保存用户账号、校园认证与信用信息
item	owner_id, category_id, mode, price, deposit, status	保存物品基础信息、价格策略和可用状态
order	buyer_id, seller_id, order_type, amount, deposit_amount, status	统一记录交易单与租赁单
lease_record	order_id, start_time, end_time, return_time, inspection_result	描述租期、归还和验收过程
payment_record	order_id, pay_type, pay_status, transaction_no, callback_time	记录支付与退款流水
review	order_id, from_user_id, to_user_id, score, content	存储交易后的互评数据
message	session_id, sender_id, receiver_id, message_type, content	支持即时沟通和系统通知
credit_record	user_id, change_type, score_delta, reason	跟踪信用变动过程，支撑风控决策

4.3 数据一致性与索引设计

订单、支付和租赁记录之间存在强关联关系，因此需要在事务边界内保证状态同步，避免出现支付成功但订单未更新或退款执行后押金未释放的问题。在索引层面，可针对物品类别、物品状态、发布时间、订单状态、用户编号和校园区域等高

频检索字段建立组合索引。对于首页推荐、热门分类和公告等读多写少的数据，则优先使用 Redis 缓存以降低数据库访问压力。

5 关键模块设计与实现

5.1 用户认证与校园身份核验

系统采用微信授权登录作为用户接入入口，并结合校园身份核验机制提升账号真实性。用户首次登录后获取 `openid` 并生成平台账户，随后补充学号、院系、校区等信息，通过人工审核或第三方校验方式确认身份状态。只有完成身份核验的用户才能发布物品或发起高风险交易。这种分级权限设计既控制了平台风险，也与校园业务场景相匹配。

5.2 物品发布与审核模块

物品发布页面要求用户填写标题、分类、成色、租赁价格、出售价格、押金、所在校区和图片等信息，并支持草稿保存与上下架管理。后台审核模块对敏感词、违规图片和错误分类进行处理。由于物品详情会同时影响搜索体验和下单决策，因此系统在数据结构上保留了多图展示、标签扩展和状态控制字段，以便后续引入推荐算法或机器审核能力。

5.3 租赁与交易统一订单模型

统一订单模型是本系统的核心设计之一。系统通过 `order_type` 区分“LEASE”和“SALE”两类业务，在公共字段中保存买卖双方、订单金额、支付状态和评价状态，在扩展字段或关联表中记录租期、押金、归还时间和验收结果。这样既保证了交易流程的共性复用，又保留了租赁业务所需的特殊节点。对于时间段冲突检测，可在创建租赁订单前查询目标物品在指定时间范围内是否存在未完成租赁记录，并配合数据库锁或乐观锁机制减少并发超卖。

5.4 消息通知与信用评价

交易双方在订单生成后往往需要就交付时间、地点和物品状态进行沟通，因此系统基于 WebSocket 实现实时通知与会话消息推送。平台消息包括订单状态变更、支付结果、到期提醒和审核通知等，用户消息则面向点对点协商。评价模块在订单

完成后开放，支持星级评分和文字评价，并将多维度评价结果沉淀到信用记录中，用于后续的免押资格判断、风险预警和违规处罚。

5.5 安全控制与异常处理

在安全设计方面，系统通过 Spring Security 与 JWT 机制实现接口级访问控制，对关键接口进行身份验证和权限拦截；支付回调采用签名校验与幂等处理，避免重复记账；图片上传和文本输入加入文件类型校验、大小限制和敏感内容过滤；对于异常订单，后台能够发起人工仲裁并调整信用记录。通过技术控制与平台规则协同，可以显著降低刷单、恶意占用和虚假发布等风险。

6 系统测试与性能优化

6.1 测试策略

系统测试可以分为接口测试、前端交互测试、业务流程测试和性能测试四个层次。接口测试重点覆盖登录鉴权、物品发布、订单创建、支付回调和消息推送等核心 API；前端交互测试关注小程序页面路由、表单校验、列表刷新和状态反馈；业务流程测试则围绕租赁闭环、交易闭环、异常退款和信用评价进行场景验证；性能测试主要关注首页查询、订单提交和热点分类统计等高频接口在并发场景下的响应时间与稳定性。

6.2 预期测试结果分析

从项目目标出发，系统应能够在正常校园网络环境下实现较快的页面首屏加载和稳定的订单流转处理。首页商品列表、分类筛选和订单详情等典型查询接口应保持较低延迟；订单创建和支付回调必须保证事务一致性；WebSocket 消息通知应满足订单状态变化后的及时触达要求。若后续用于正式论文提交，应补充实际压测曲线、数据库执行计划截图和关键页面测试截图，以增强结论的客观性。

6.3 性能优化方案

针对潜在高并发场景，系统可从数据库索引、缓存策略、异步化和限流降级四个方面进行优化。首先，对物品检索、订单状态和用户维度建立合理索引；其次，将首页推荐、类别字典、公告和热点商品等读多写少的数据缓存到 Redis；再次，

将消息推送、统计汇总和日志记录等非关键路径任务异步化；最后，在支付回调和下单接口中引入幂等控制与频率限制，避免突发流量对核心服务造成冲击。

7 结论与展望

本文围绕校园个人物品租赁与交易场景，设计并给出了一套基于微信小程序的系统实现方案。研究从校园场景的信任建立、资源共享需求和双模式业务特征出发，提出了统一订单模型、信用评价机制和后台监管能力，并在技术层面完成了前端、后端和数据层的协同设计。该系统不仅能够满足学生群体对短期租赁和二手交易的日常需求，也具有较好的可扩展性，能够继续演进到更丰富的校园服务生态。

后续工作可以从三个方面继续深化：一是结合真实运营数据引入个性化推荐算法，提高物品曝光效率；二是完善风险控制模型，细化不同品类、不同用户等级下的押金与信用策略；三是补充更完整的实测数据、用户访谈和可用性评估，进一步增强论文的研究深度与工程说服力。

参考文献

- [1] 王珊, 萨师煊. 数据库系统概论[M]. 北京: 高等教育出版社, 2019.
- [2] Ian Sommerville. Software Engineering[M]. 10th ed. Boston: Pearson, 2015.
- [3] Kleppmann M. Designing Data-Intensive Applications[M]. Sebastopol: O'Reilly Media, 2017.
- [4] Spring. Spring Boot Reference Documentation[EB/OL].
<https://spring.io/projects/spring-boot>.
- [5] Oracle. Java Platform, Standard Edition Documentation[EB/OL].
<https://docs.oracle.com/en/java/>.
- [6] Oracle. MySQL 8.0 Reference Manual[EB/OL]. <https://dev.mysql.com/doc/>.
- [7] Redis. Redis Documentation[EB/OL]. <https://redis.io/docs/>.
- [8] Baomidou. MyBatis-Plus 官方文档[EB/OL]. <https://baomidou.com/>.
- [9] 微信开放社区. 微信小程序开发文档[EB/OL].
<https://developers.weixin.qq.com/miniprogram/dev/framework/>.
- [10] 腾讯位置服务. Webservice API 与地图服务文档[EB/OL]. <https://lbs.qq.com/>.

附录 A 论文改写建议

本参考稿已经根据仓库现有说明文档整理出较完整的论文结构，但若用于正式提交，仍建议结合学校模板进行二次加工：第一，替换封面、摘要格式、字体字号、页眉页脚和章节编号样式；第二，补充系统截图、数据库 E-R 图、接口时序图和测试结果表格；第三，将文中的“预期效果”改写为基于真实开发与测试结果的客观结论；第四，根据指导老师要求补充国内外研究现状、数据来源和参考文献格式规范。